

챕터 4. Kubernetes 시작하기

회의실을 나선 오픈이의 머릿속은 복잡했습니다. 시연은 문제없이 끝났지만, 팀장이 마지막에 던진 질문이 마음에 걸렸습니다. 운영 중에 컨테이너가 죽으면 어떻게 되살릴 것인가. 도커만으로는 그 답이 보이지 않았습니다.

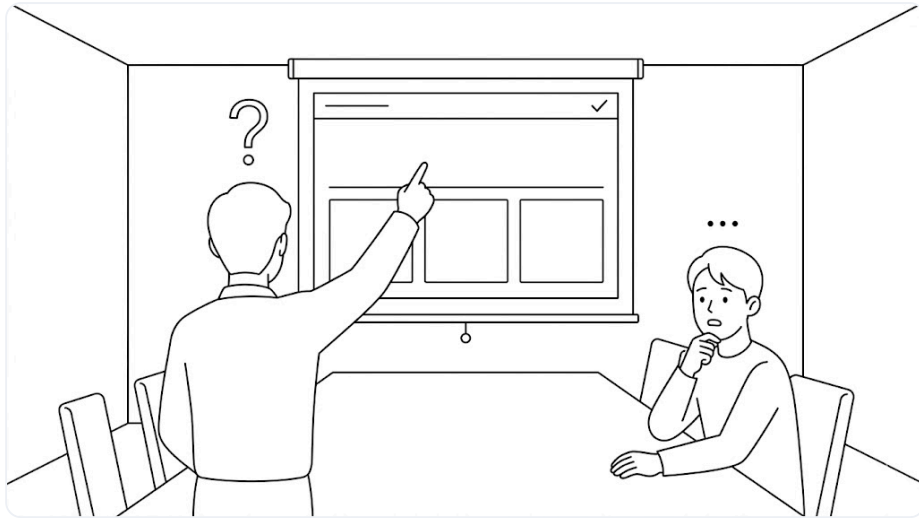


그림 4-1. 팀장의 운영 질문에 답이 막힌 순간

점심을 먹고 돌아오는 길에 선배에게 고민을 털어놨습니다.

오픈이 : "도커를 공부하면 끝일 줄 알았는데, 운영에서 컨테이너가 죽거나 트래픽이 몰릴 때 어떻게 해야 할지 모르겠어요."

선배 : "도커만으로는 운영까지 감당하기 어려워요. 그래서 쿠버네티스를 같이 알아야 해요. 컨테이너에 대한 관리는 쿠버네티스 담당이거든요."

선배의 말을 들은 오픈이는 퇴근 후 집에서 노트북을 열었습니다. 그리고 쿠버네티스가 무엇이고, 어떤 역할을 하는지 살펴보기로 했습니다.

이번 챕터에서 다룰 쿠버네티스 리소스의 전체 구조는 다음과 같습니다.

챕터 4 한눈에 보기 — 전체 구조



그림 4-2. 챕터 4 한눈에 보기 - 전체 구조

준비하기

Minikube 설치

쿠버네티스 실습에 쓰는 로컬 클러스터 도구 Minikube를 설치합니다. OS에 맞는 명령을 터미널에서 실행합니다.

터미널 Minikube 설치

```
# macOS
brew install minikube

# Windows
winget install Kubernetes.minikube
```

예제 코드

이 책의 실습 코드는 GitHub 레포 하나에 챕터별 폴더로 담겨 있습니다. 아래 명령으로 레포를 한 번 git clone 합니다. 이후 챕터마다 해당 폴더로 이동해 실습합니다.

터미널 예제 코드 클론

```
git clone https://github.com/metacoding-10-linux-docker/start
```

이번 챕터는 `ex08` ~ `ex10` 폴더를 사용합니다.

완성된 코드는 아래 주소에서 확인하세요.

터미널 완성 코드

```
https://github.com/metacoding-10-linux-docker/final
```

4.1 Kubernetes - 원하는 상태를 유지하기

4.1.1 도커는 실행 명령, Kubernetes는 약속 유지

도커와 쿠버네티스는 컨테이너를 다루는 방식이 다릅니다.

- **Docker(명령형):** "이 컨테이너를 실행해라."
- **Kubernetes(선언형):** "이 서비스를 원하는 상태로 유지해라."

이 선언형 방식은 프랜차이즈 본사가 가맹점을 관리하는 방식과 비슷합니다.

본사는 가맹점을 직접 운영하지 않습니다. 대신 가맹점 수나 모습 같은 **규칙**을 정하고 그 규칙이 지켜지는지를 관리합니다. 만약 매장이 문을 닫으면 **새 매장을 다시 열어** 개수를 맞추고, 수요가 많아지면 **새로운 매장을** 추가합니다.

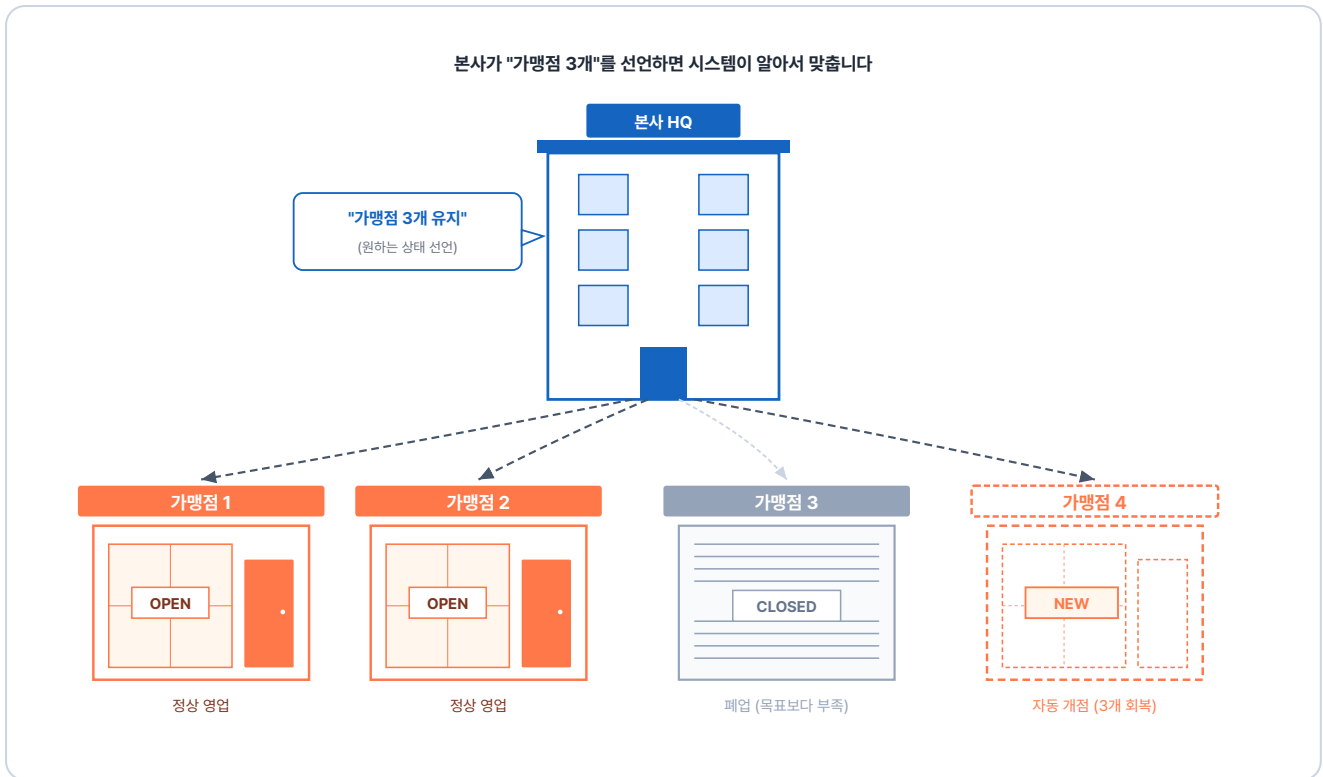


그림 4-3. 본사가 "가맹점 3개 유지"를 선언하면 시스템이 가맹점 개수를 자동으로 맞추는 구조

쿠버네티스도 마찬가지입니다. 컨테이너 수나 상태 규칙 등 **원하는 상태**를 선언해 두면, 시스템이 자동으로 그 상태를 **유지**합니다.

4.1.2 컨트롤 플레인과 워커 노드

쿠버네티스는 크게 **컨트롤 플레인(Control Plane)** 과 **워커 노드(Worker Node)** 로 이루어집니다.

노드(Node): 컨테이너가 실제로 돌아가는 컴퓨터 한 대를 말합니다. 실제 서비스 환경에서는 수십~수백 대의 노드(서버)가 모여 하나의 **클러스터(Cluster)** 를 이룹니다. 컨트롤 플레인과 워커 노드 모두 이런 노드 위에서 동작합니다.

컨트롤 플레인은 **프랜차이즈 본사 관리팀**처럼 컨테이너를 관리할 **규칙을 정하고 지시를 내립니다**. 지시를 받은 워커 노드가 실제로 컨테이너를 **실행하고 관리**합니다.

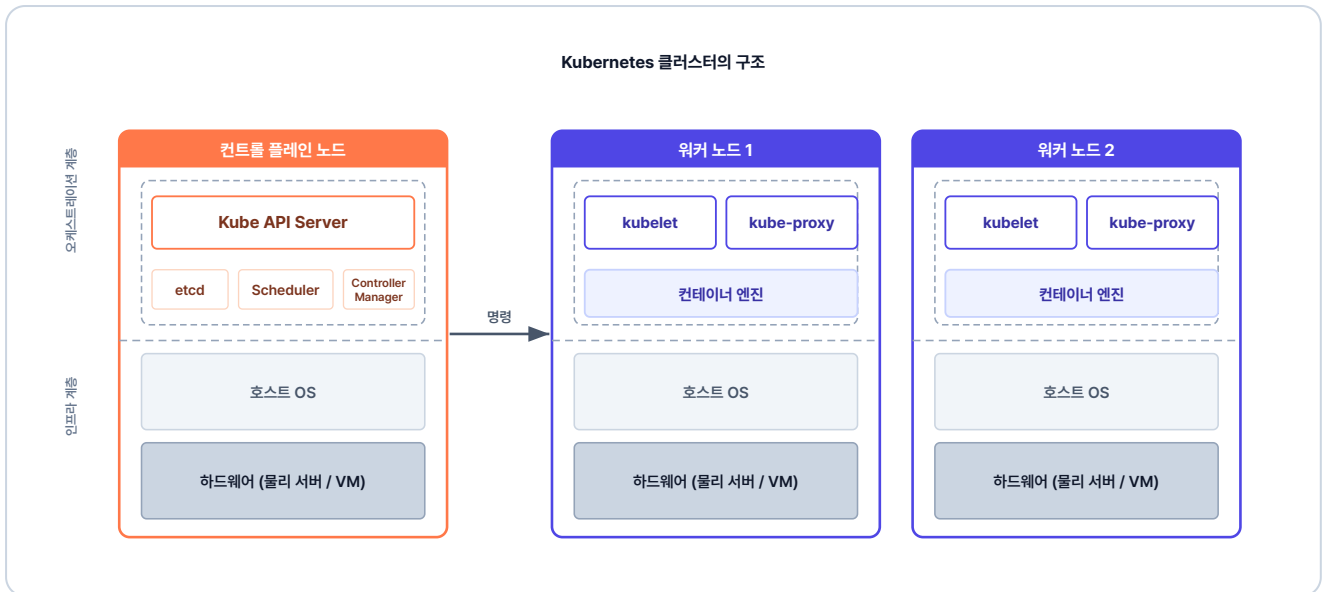


그림 4-4. Kubernetes 클러스터의 구조

쿠버네티스는 컨테이너를 Pod라는 단위로 묶어 관리합니다.

4.1.3 Kubernetes의 동작 원리

개발자가 명령을 내려 Pod 하나가 만들어지는 일은 프랜차이즈 본사가 새 가맹점을 여는 흐름과 비슷합니다. 크게 **명령** → **판단** → **지시** 세 단계로 일어납니다. 하나씩 들여다보겠습니다.

1단계 - 개발자가 클러스터에 명령을 보낸다

개발자가 클러스터에 명령을 보내는 일은 프랜차이즈 본사에 가맹점을 신청하는 것과 같습니다. 외부에서 들어오는 모든 요청은 **가장 먼저** 컨트롤 플레인 내부의 **Kube API Server**를 통과합니다.

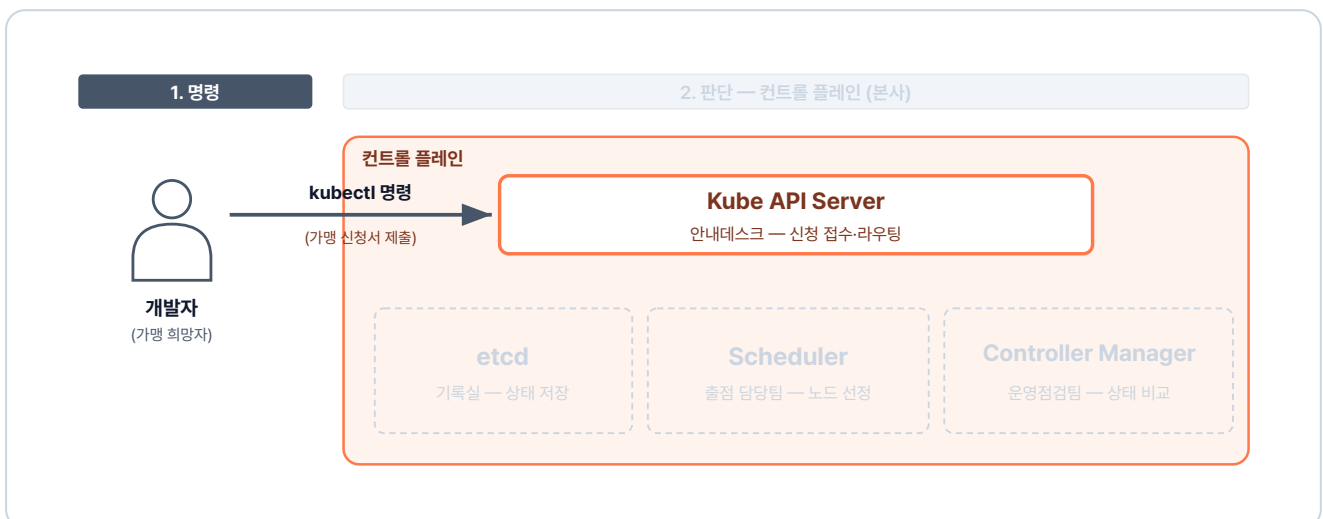


그림 4-5. 명령 단계 — 개발자의 명령이 Kube API Server에 도착하는 길

2단계 - 컨트롤 플레인이 명령을 어떻게 처리할지 판단한다

본사는 신청을 받으면 **새 매장의 위치**를 정하고 **기존 매장들의 상태**를 점검합니다. 이 과정에서 컨트롤 플레인의 구성 요소는 **Kube API Server**를 통해 서로 상태를 주고받으며 **이 명령을 어떻게 처리할지** 결정합니다.

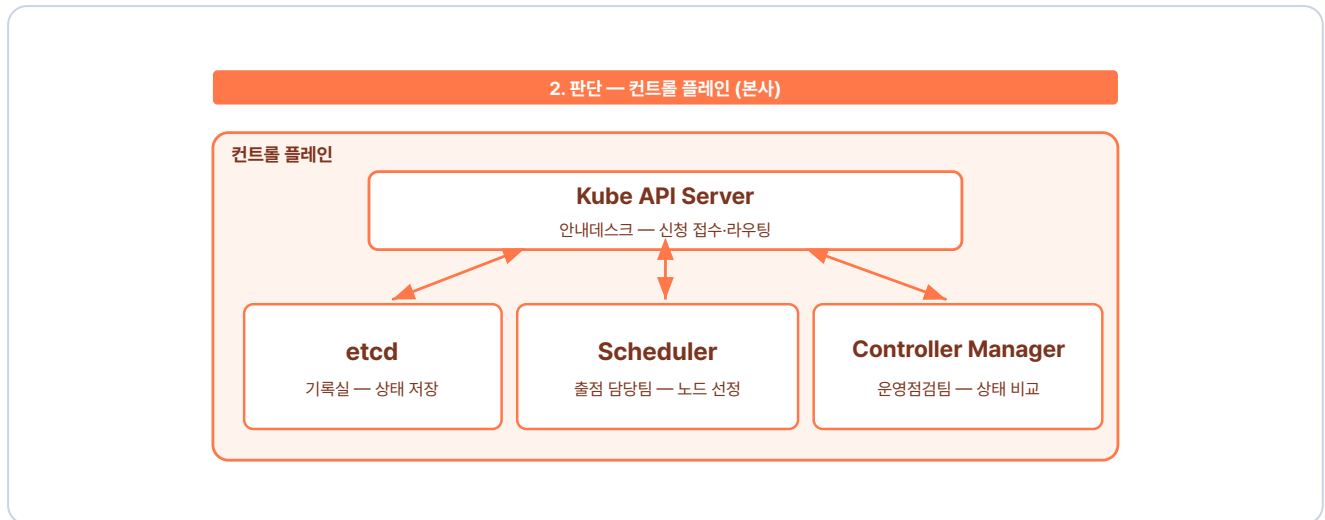


그림 4-6. 판단 단계 — Kube API Server를 중심으로 구성 요소들이 협의하는 모습

Kube API Server는 들어온 요청을 바탕으로 **클러스터의 상태**를 업데이트하고, 이 정보를 **etcd**에 보관합니다. **Scheduler**는 보관된 정보를 확인해 새 Pod를 **어느 노드에 배치할지** 정합니다. 동시에 **Controller Manager**는 클러스터 상태가 **원래 설정된 모습대로 유지되고 있는지** 감시합니다.

3단계 - 컨트롤 플레인이 워커 노드 kubelet에 지시한다

판단이 완료되면 **Kube API Server**가 워커 노드의 **kubelet**에게 지시를 내립니다. 지시를 받은 **kubelet**은 노드 안에서 컨테이너를 실행하고, 그 상태가 정의된 모습에서 벗어나지 않게 **계속 점검합니다**.

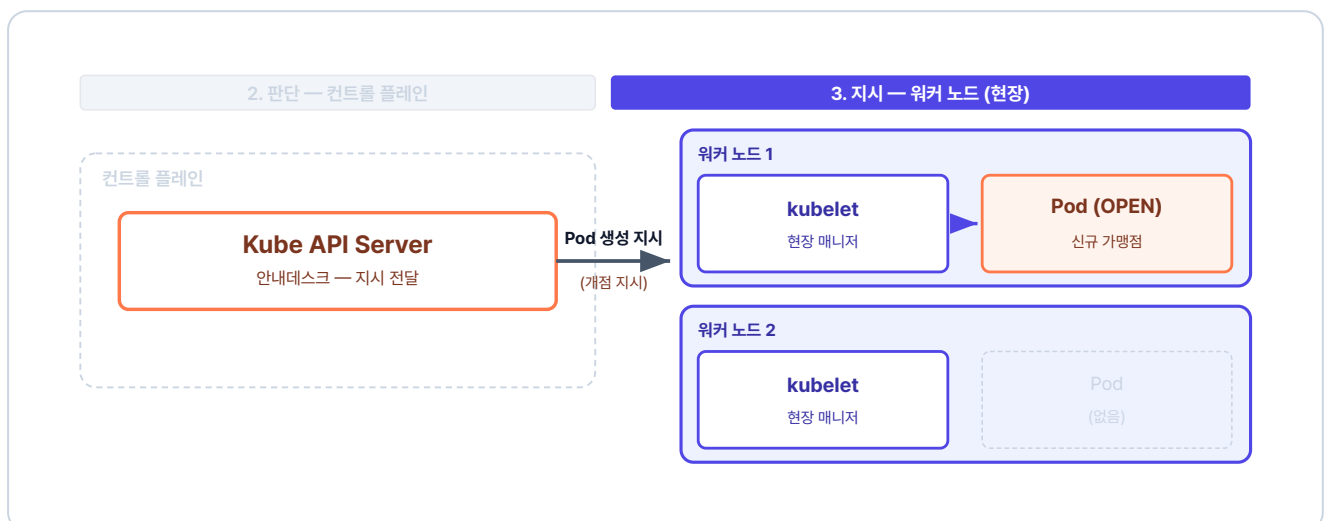


그림 4-7. 지시 단계 — kubelet이 컨트롤 플레인의 지시를 받아 Pod를 띄우는 모습

이런 명령 → 판단 → 지시를 거쳐 **Pod**가 생성됩니다.

4.1.4 Kubernetes 핵심 리소스 한눈에

쿠버네티스 세상에서는 작업 단위를 **리소스(Resource)** 라고 부릅니다. 사용자의 요청은 여러 리소스를 거쳐 실제 컨테이너에 도달합니다. 전체적인 흐름은 다음과 같습니다.

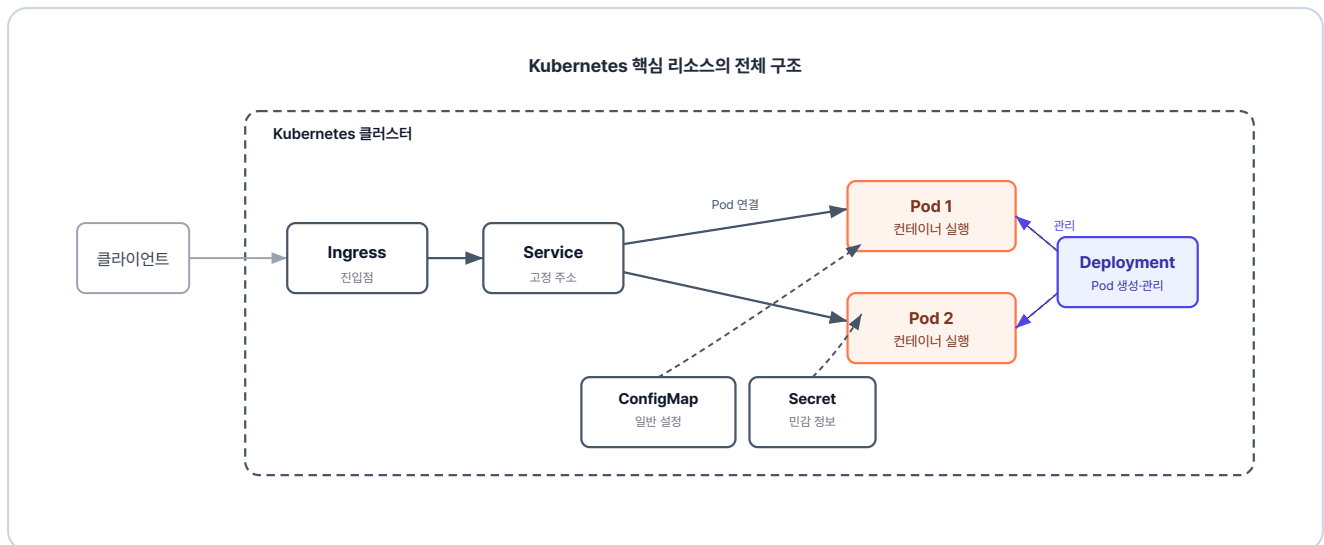


그림 4-8. Kubernetes 핵심 리소스의 전체 구조

이번 챕터에서는 가장 기본이 되는 Pod와 Deployment를 먼저 다뤄 보겠습니다.

4.2 Minikube - 로컬에 만드는 작은 클러스터

4.2.1 노트북 한 대로 클러스터 흥내 내기

컨트롤 플레인과 워커 노드를 모두 갖추려면 서버가 여러 대 필요해 보이지만, 연습 단계에서는 노트북 한 대로 충분합니다. Minikube를 쓰면 내 컴퓨터 안에 쿠버네티스 환경을 구축할 수 있습니다.

Minikube는 **노드 하나로 동작합니다**. 이 노드가 컨트롤 플레인 역할과 워커 노드 역할을 함께 수행합니다.

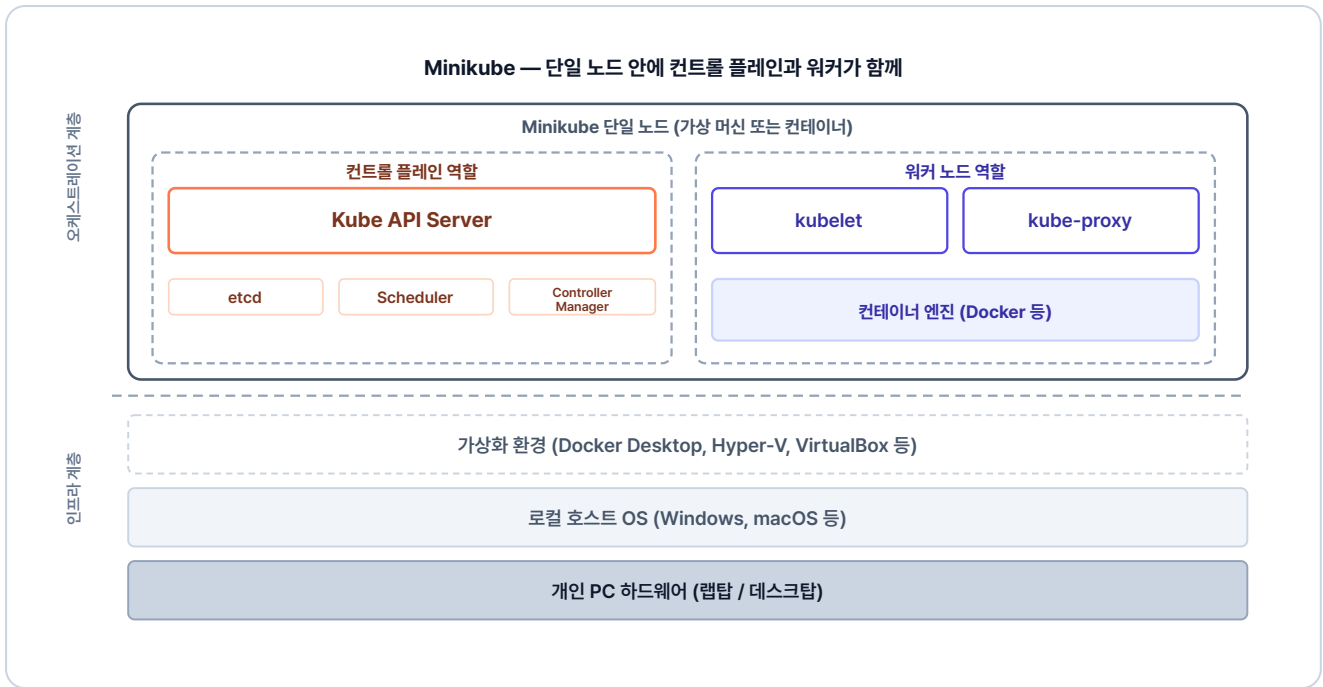


그림 4-9. Minikube는 단일 노드 안에 컨트롤 플레인과 워커 노드 기능이 함께 들어간 구조

이렇게 구조가 단순해 리소스도 적게 사용합니다. Minikube에서 작성한 설정 파일은 실제 운영 환경에서도 그대로 사용할 수 있습니다.

'이제 Minikube를 직접 실행해 봐야겠다.'

4.2.2 클러스터 시작

앞의 준비하기를 참고해 Minikube를 먼저 설치합니다. 그 다음 아래 명령으로 노트북 안에 클러스터를 시작합니다.

터미널 클러스터 시작

```
minikube start # 클러스터 시작
```

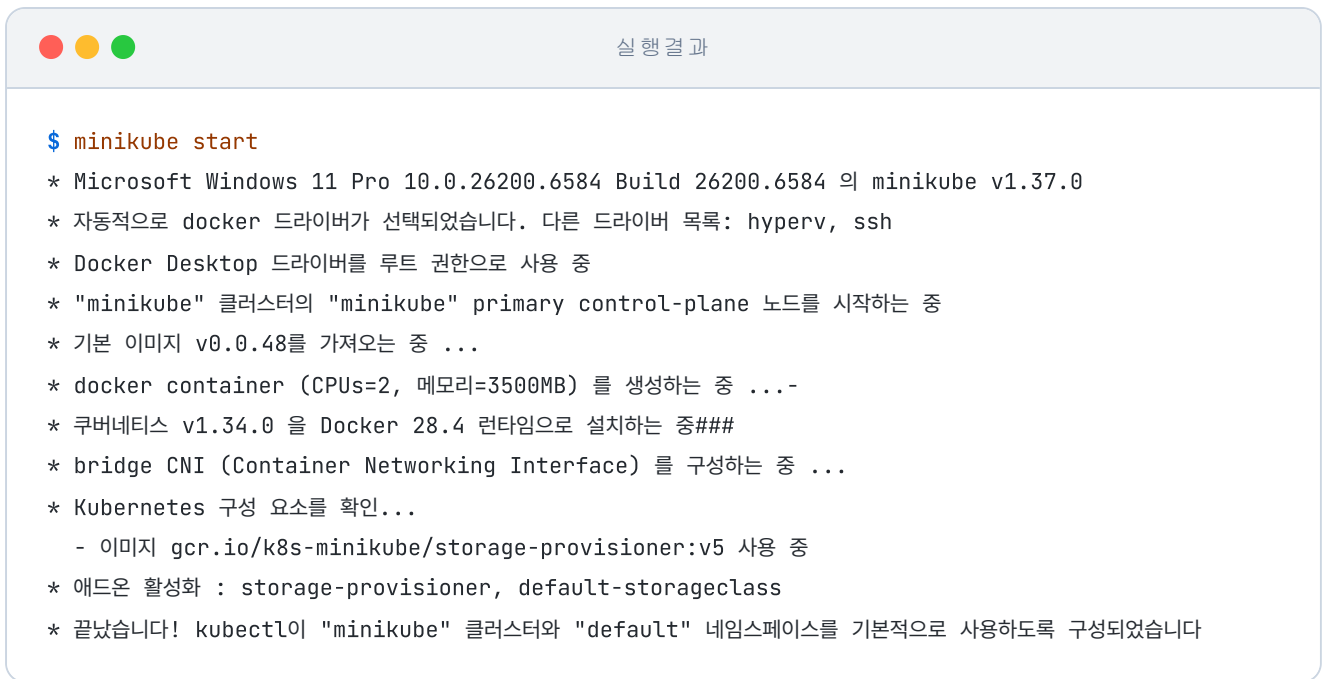


그림 4-10. minikube start 실행 결과

4.2.3 자주 쓰는 Minikube 명령어

실습에서 자주 쓰게 될 명령어는 다음과 같습니다.

명령어	설명
<code>minikube start</code>	클러스터를 시작합니다.
<code>minikube stop</code>	실행 중인 클러스터를 중지합니다.
<code>minikube status</code>	현재 클러스터의 구동 상태를 확인합니다.
<code>minikube service <서비스명> --url</code>	생성한 서비스에 접근할 수 있는 URL을 생성합니다.

4.3 Pod - Kubernetes의 최소 단위

Minikube 환경을 구성했으니 이제 실제 쿠버네티스의 리소스를 하나씩 생성해 볼 차례입니다. 도커만 쓸 때와 무엇이 다른지, 쿠버네티스라는 시스템이 컨테이너를 다루는 최소 단위부터 확인해 보겠습니다.

4.3.1 Pod란?

Pod(파드)는 하나 이상의 컨테이너로 구성된 단위입니다. 쿠버네티스는 컨테이너를 하나씩 직접 다루지 않고, 이 Pod를 단위로 관리합니다.



그림 4-11. Pod의 구조

왜 컨테이너 대신 Pod를 쓸까요

컨테이너는 서로 독립적으로 동작해 네트워크·스토리지·생명주기를 함께 관리하기 어렵습니다. Pod는 이를 다음과 같이 해결합니다.

- **하나처럼 묶어서 관리:** 여러 컨테이너를 하나로 묶어 같이 생성·종료되고, 함께 동작하도록 합니다.
- **네트워크 공유:** Pod 내부 컨테이너들은 하나의 IP를 공유해 같은 서버 안에서 통신합니다.
- **관리 기준 단위:** 스케줄링, 확장, 장애 복구 등은 컨테이너가 아니라 Pod 기준으로 이루어집니다.

그래서 쿠버네티스는 컨테이너가 아닌 Pod를 '배포·운영의 최소 단위'로 관리합니다.

4.3.2 명령어로 Pod 만들기

클러스터에 명령을 내릴 때 사용하는 도구는 **kubectl**입니다. "이런 상태를 만들어 달라"고 쿠버네티스에 요청하는 명령어입니다.

가장 먼저 시도해 볼 방식은 도커와 비슷한 명령어 기반의 생성입니다.

터미널 명령어로 첫 Pod 생성

```
kubectl run hello-pod1 --image=nginx # nginx 이미지로 Pod 생성
```

실행 결과

```
$ kubectl run hello-pod1 --image=nginx
pod/hello-pod1 created
```

그림 4-12. kubectl run으로 Pod 생성

Pod가 생성되었습니다. 이 방식은 `docker run` 과 비슷한 방식입니다.

'이건 명령형 방식이네. 그럼 선언형으로는 어떻게 만들까?'

4.3.3 YAML로 Pod 만들기

선언형은 원하는 상태를 YAML 파일로 작성하여 클러스터에 반영하는 방식입니다. 명령어로 일일이 지시하는 대신, 쿠버네티스가 YAML을 읽어 원하는 상태로 리소스를 생성합니다.

실습 코드는 `ex08` 폴더에 있습니다.

```
ex08/
├─ hello-pod2.yml    # nginx Pod 정의
└─ README.md        # 실습 안내
```

다음은 nginx 이미지로 Pod 하나를 만드는 YAML입니다.

실습 1 ex08/hello-pod2.yml. nginx Pod 정의

```
apiVersion: v1
kind: Pod                # 리소스 종류
metadata:
  name: hello-pod2       # 리소스명
spec:
  containers:
    - name: hello-container
      image: nginx:1.20   # 이미지
```

이렇게 작성한 파일을 클러스터에 적용할 때는 **kubectl apply** 명령어를 사용합니다.

터미널 YAML로 Pod 생성

```
kubectl apply -f ex08/hello-pod2.yaml # YAML로 Pod 생성
```



실행 결과

```
$ kubectl apply -f ex08/hello-pod2.yaml  
pod/hello-pod2 created
```

그림 4-13. kubectl apply로 Pod 생성

YAML 파일을 통해 Pod가 생성되었습니다.

4.3.4 Pod 조회

Pod 두 개를 만들었으니 이제 목록을 조회해 보겠습니다.

터미널 Pod 목록 조회

```
kubectl get pod # Pod 목록 조회
```



실행 결과

```
$ kubectl get pod  
NAME READY STATUS RESTARTS AGE  
hello-pod1 1/1 Running 0 8m39s  
hello-pod2 1/1 Running 0 23s
```

그림 4-14. Pod 목록 조회 결과

Pod의 상태나 컨테이너, IP 같은 상세 정보를 보고 싶다면 **kubectl describe** 명령어를 사용합니다.

터미널 Pod 상세 정보 조회

```
kubectl describe pod hello-pod2 # Pod 상세 정보 조회
```

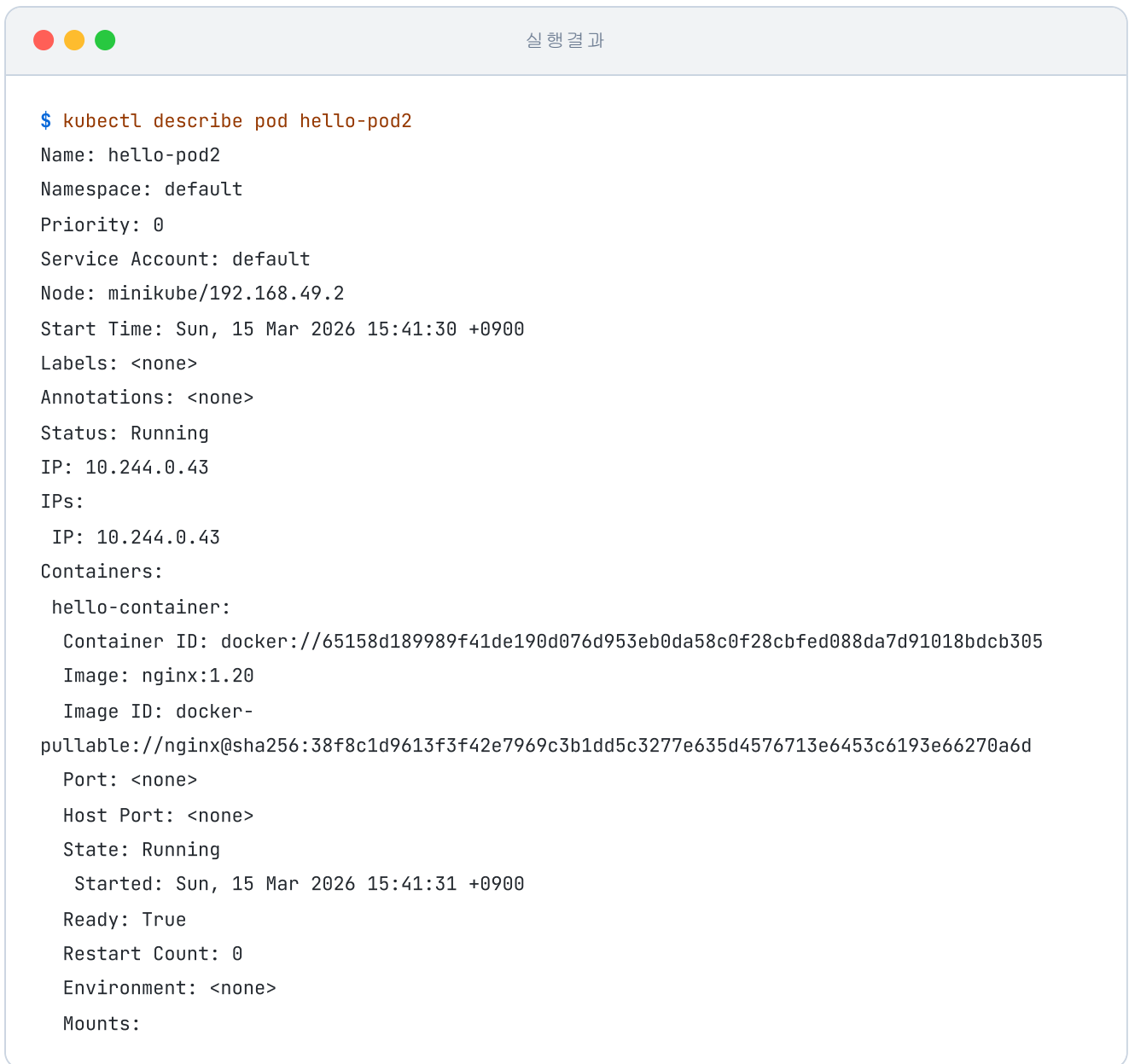


그림 4-15. Pod 상세 조회

4.3.5 자주 쓰는 kubectl 명령어

실습에서 자주 쓰게 될 기본 명령어는 다음과 같습니다.

명령어	설명
<code>kubectl apply -f <파일></code>	YAML로 리소스 생성/업데이트
<code>kubectl get <리소스></code>	리소스 목록 조회
<code>kubectl describe <리소스> <이름></code>	리소스 상세 정보

명령어	설명
<code>kubectl delete <리소스> <이름></code>	리소스 삭제
<code>kubectl exec -it <Pod명> -- bash</code>	Pod 내부 접속
<code>kubectl logs <Pod명></code>	Pod 로그 확인

Pod를 생성한 오픈이는 이런 의문이 들었습니다.

'YAML로 정의해 뒀으니, 이제 쿠버네티스가 알아서 관리해 주는 걸까?'

4.4 Deployment - 자동 복구·스케일링·무중단 배포

4.4.1 Pod 하나를 직접 만들면 생기는 문제

방금 만든 Pod로 실험을 해 보겠습니다. 운영 중에 누군가의 실수로 Pod가 삭제되거나, 프로그램 오류로 종료된다면 어떤 일이 벌어질까요.

YAML로 생성한 hello-pod2를 삭제해 보겠습니다.

터미널 Pod 삭제 후 자동 복구 여부 확인

```
kubectl delete pod hello-pod2    # hello-pod2 삭제
kubectl get pod                  # 남은 Pod 목록 확인
```

실행 결과

```
$ kubectl delete pod hello-pod2
pod "hello-pod2" deleted
$ kubectl get pod
NAME READY STATUS RESTARTS AGE
hello-pod1 1/1 Running 0 19m
```

그림 4-16. Pod 삭제 후 목록을 조회하면 hello-pod2가 사라져 있습니다

hello-pod2는 목록에서 사라졌고, 시간이 지나도 다시 나타나지 않았습니다.

'아, 그냥 선언만 해서는 자동 복구가 안 되네.'

Pod 자체는 사라져도 자동으로 복구되지 않습니다. 그래서 Pod를 원하는 상태로 계속 유지하는 리소스가 필요합니다. 그 리소스가 쿠버네티스의 **Deployment(디플로이먼트)** 입니다.

Deployment: 파드(Pod)에 대한 선언적 업데이트를 제공하는 쿠버네티스 리소스입니다. 원하는 상태를 선언해 두면 시스템이 그 상태를 자동으로 유지하며, 자동 복구(Self-healing)·롤링 업데이트·롤백·스케일링까지 함께 담당합니다.

4.4.2 Deployment - 원하는 상태를 유지하는 매뉴얼

Deployment는 "Pod를 몇 개 유지할지, 문제가 생기면 어떻게 교체할지"를 적어 두는 매뉴얼과 같습니다. 선언한 상태와 실제 상태를 비교해, 차이가 생기면 스스로 조정해 원하는 상태로 되돌립니다.

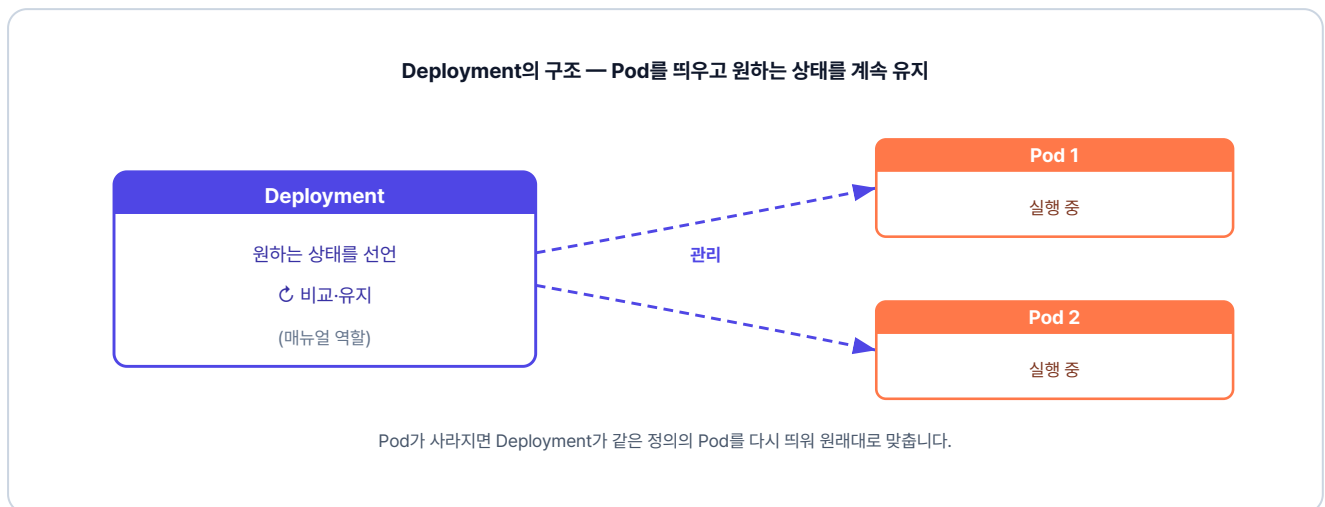


그림 4-17. Deployment의 구조 — Pod를 띄우고 원하는 상태를 계속 유지

실습 코드는 `ex09` 폴더에 있습니다.

```
ex09/  
├─ deploy-ex01.yml # nginx Pod 1개를 유지하는 Deployment 정의  
└─ README.md      # 실습 안내
```

다음은 nginx Pod 한 개를 항상 유지하도록 정의한 Deployment YAML입니다.

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deploy
spec:
  replicas: 1          # pod에 대한 상태 지정
  selector:
    matchLabels:
      app: nginx        # 'app: nginx' 라벨이 붙은 Pod를 내 관리 대상으로 지정
  template:
    metadata:
      labels:
        app: nginx      # pod에 붙일 라벨
    spec:
      containers:
        - name: nginx-container
          image: nginx:1.20

```

여기서 중요한 개념은 **selector** 와 **labels** 의 연결입니다.

Deployment가 관리할 Pod를 생성할 때, Pod의 **labels** 영역에 이름표를 붙입니다. 그리고 Deployment의 **selector** 에 똑같은 이름표를 지정해 두면, 쿠버네티스는 그 이름표를 가진 Pod들만 식별하여 관리합니다.

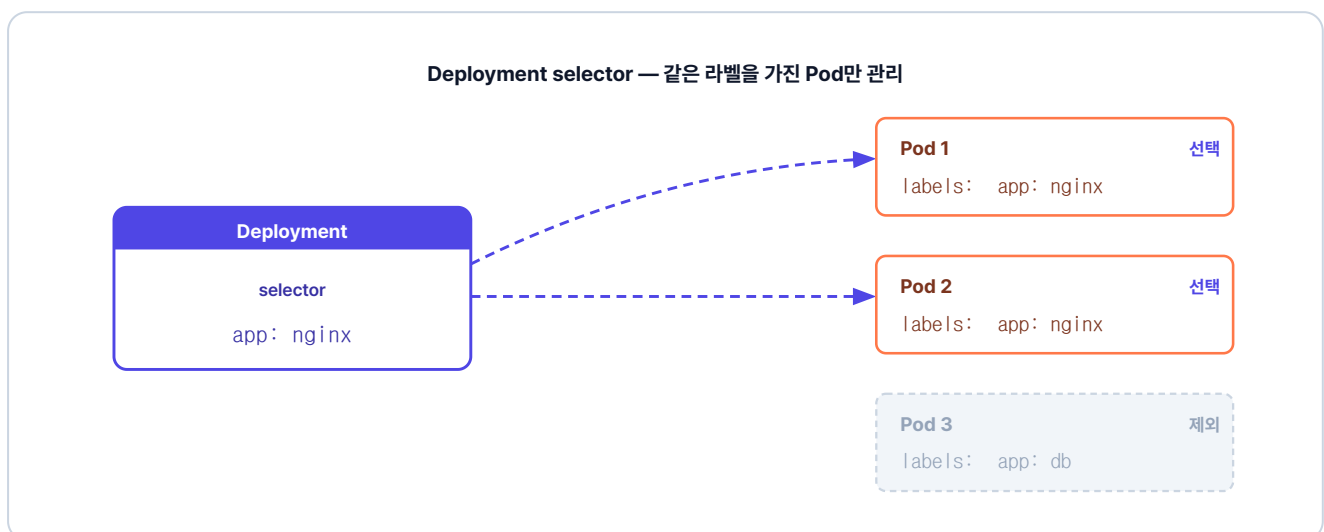


그림 4-18. selector가 지정한 라벨(app: nginx)을 가진 Pod만 골라 관리, 다른 라벨은 건드리지 않음

작성한 파일을 적용해 보겠습니다.

터미널**Deployment 생성**

```
kubectl apply -f ex09/deploy-ex01.yml # Deployment 생성
kubectl get pod                       # Pod 목록
```



실행 결과

```
$ kubectl get pod
NAME READY STATUS RESTARTS AGE
hello-pod1 1/1 Running 0 19m
nginx-deploy-756b46b54c-8q8p1 1/1 Running 0 5s
```

그림 4-19. Deployment가 만든 Pod가 뜬 모습

이제 Deployment가 정말로 원하는 상태를 유지하는지 확인하기 위해, 현재 실행 중인 Pod를 전부 지웁니다.

터미널**Pod 전체 삭제 후 자동 복구 확인**

```
kubectl delete pod --all # 모든 Pod 삭제
kubectl get pod          # 목록 재확인
```



실행 결과

```
$ kubectl delete pod --all
pod "hello-pod1" deleted
pod "nginx-deploy-756b46b54c-8q8p1" deleted
$ kubectl get pod
NAME READY STATUS RESTARTS AGE
nginx-deploy-756b46b54c-2sv8x 1/1 Running 0 4s
```

그림 4-20. Pod를 다 지워도 Deployment가 자동으로 새 Pod를 띄웁니다

hello-pod1은 그대로 사라졌지만 Deployment가 만든 Pod는 삭제된 후 새 Pod로 다시 실행되었습니다.

'팀장님이 물어봤던 게 이거였구나. 원하는 상태만 정해 두면, 그다음은 Deployment가 자동으로 관리해 주네.'

4.4.3 ReplicaSet - Pod 개수 자동 관리

운영 환경에서는 부하를 분산하기 위해 Pod를 여러 개 실행합니다. 이때 Pod의 개수를 일정하게 유지하는 리소스가 바로 **ReplicaSet(레플리카셋)** 입니다. 사용자가 선언한 개수와 실제 작동 중인 Pod의 개수를 비교해, 모자라거나 남는 만큼 조절합니다.

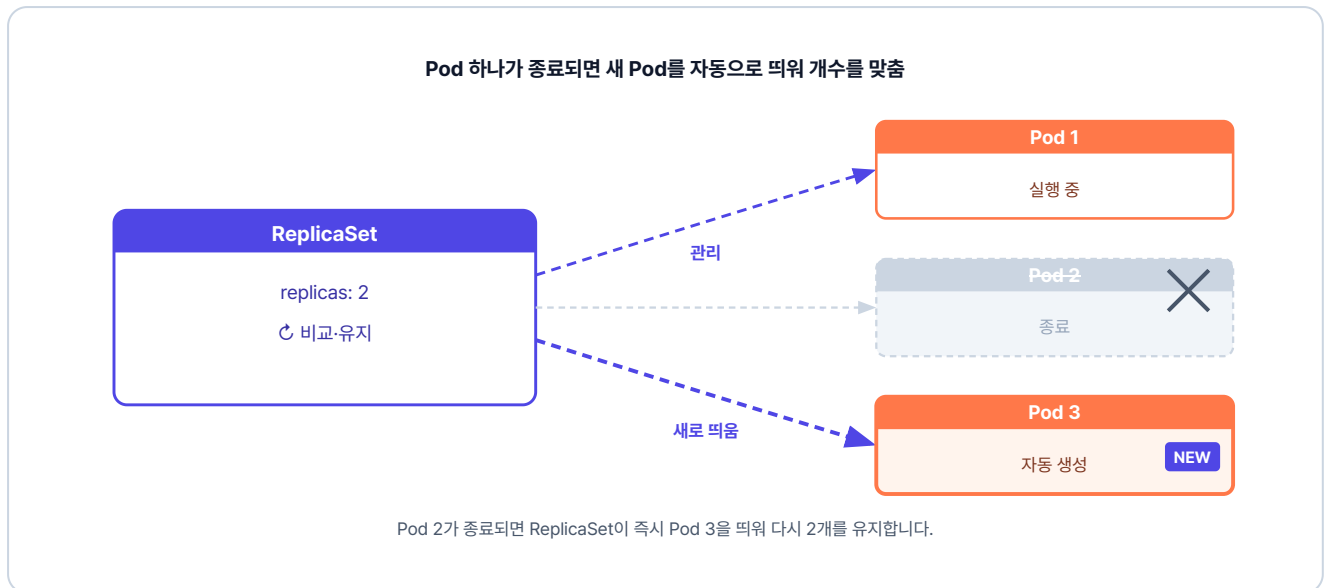


그림 4-21. Pod 하나가 종료되면 새 Pod를 자동으로 띄워 개수를 맞춤

ReplicaSet과 Deployment

ReplicaSet은 쿠버네티스에 별도로 존재하는 리소스이지만, 롤링 업데이트나 롤백 같은 배포 기능이 없습니다. 그래서 실무에서는 **ReplicaSet을 직접 만들지 않고 Deployment를 사용합니다.** Deployment의 **replicas** 속성을 통해 내부적으로 ReplicaSet을 자동으로 만들어 Pod 개수를 유지하고, 롤링 업데이트와 롤백까지 함께 제공합니다.

실습 코드는 `ex10` 폴더에 있습니다.

```
ex10/  
├─ deploy-ex02.yml # nginx Pod 4개를 유지하는 Deployment 정의  
└─ README.md      # 실습 안내
```

다음은 `replicas`를 4로 늘려 같은 Pod 4대를 동시에 띄우고 유지하는 Deployment YAML입니다.

실습 3 ex10/deploy-ex02.yml. replicas 4와 롤링 업데이트 전략

```
apiVersion: apps/v1
```

```

kind: Deployment
metadata:
  name: nginx-replica
spec:
  replicas: 4          # pod 수 지정

  strategy:
    type: RollingUpdate # 롤링 업데이트 전략
    rollingUpdate:
      maxSurge: 4      # 업데이트 중 최대 4개까지 추가 생성
      maxUnavailable: 0 # 기존 Pod를 먼저 종료하지 않음 (무중단 배포)

  selector:          # 라벨이 app: nginx 인 pod를 관리
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx    # pod에 붙일 라벨
    spec:
      containers:
        - name: nginx-container
          image: nginx:1.20

```

이 YAML 파일을 클러스터에 적용하고, 설정한 대로 4개의 Pod가 생성되는지 확인합니다.

터미널 replicas 4 적용

```

kubectl apply -f ex10/deploy-ex02.yaml # Deployment YAML 적용
kubectl get pod                        # 생성된 Pod 목록 확인

```

실행 결과

```

$ kubectl get pod
NAME READY STATUS RESTARTS AGE
nginx-replica-756b46b54c-l5l7f 1/1 Running 0 5s
nginx-replica-756b46b54c-t8fh1 1/1 Running 0 5s
nginx-replica-756b46b54c-vp5g5 1/1 Running 0 5s
nginx-replica-756b46b54c-vzv7n 1/1 Running 0 5s

```

그림 4-22. replicas 설정대로 Pod 4개가 생성

4.4.4 롤링 업데이트 - 끊김 없는 버전 교체

운영 중인 서비스를 새 버전으로 교체할 때, 여러 Pod를 수동으로 교체하면 번거로울 뿐만 아니라 그 과정에서 서비스가 중단되기 쉽습니다. 쿠버네티스의 **롤링 업데이트(Rolling Update)**는 기존 버전을 한꺼번에 내리지 않고 새 버전으로 조금씩 교체하여, **서비스가 멈추지 않는 무중단 배포**를 제공합니다.

앞서 만든 `deploy-ex02.yaml`에는 이 롤링 업데이트 동작을 정하는 `strategy` 블록이 들어 있습니다. 각 속성은 다음과 같습니다.

- **maxSurge**: 업데이트 중 replicas로 지정한 수보다 잠시 더 늘어나는 Pod의 수
- **maxUnavailable**: 업데이트 중 replicas로 지정한 수보다 잠시 줄어들어도 되는 Pod의 수

예를 들어 replicas: 4, maxSurge: 4, maxUnavailable: 0이라면, 기존 4개는 그대로 둔 상태에서 새 버전 4개를 추가합니다. 새 Pod가 준비되면 기존 Pod를 차례로 종료시킵니다.

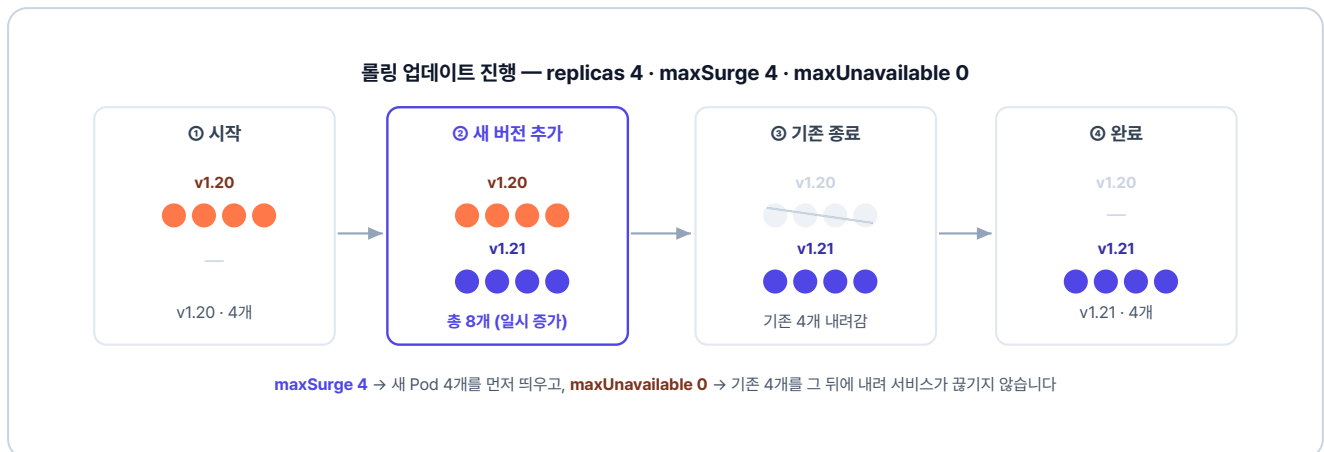


그림 4-23. 롤링 업데이트 진행 - 새 Pod 4개를 띄운 뒤 기존 Pod 4개를 차례로 내립니다

maxSurge를 줄이면 한 번에 띄울 새 Pod 수가 제한되어 천천히 교체되고, **maxUnavailable**을 늘리면 기존 Pod가 한 번에 더 많이 종료되는 것을 허용해 빠르게 교체할 수 있습니다. 이렇게 두 값을 조정해 원하는 배포 방식을 설정합니다.

다음 명령어로 이미지 버전을 교체해 보겠습니다.

터미널 이미지 버전 교체 (롤링 업데이트 트리거)

```
# 이미지를 nginx:1.21로 교체
kubectl set image deployment/nginx-replica nginx-container=nginx:1.21
```

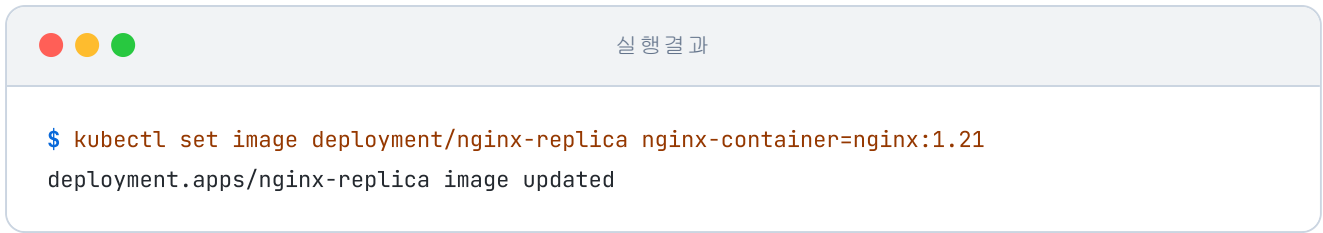


그림 4-24. 이미지 버전 업데이트 실행

`kubectl get` 명령어에 `-w` 옵션을 추가하면 Pod 상태 변화를 실시간으로 볼 수 있습니다.

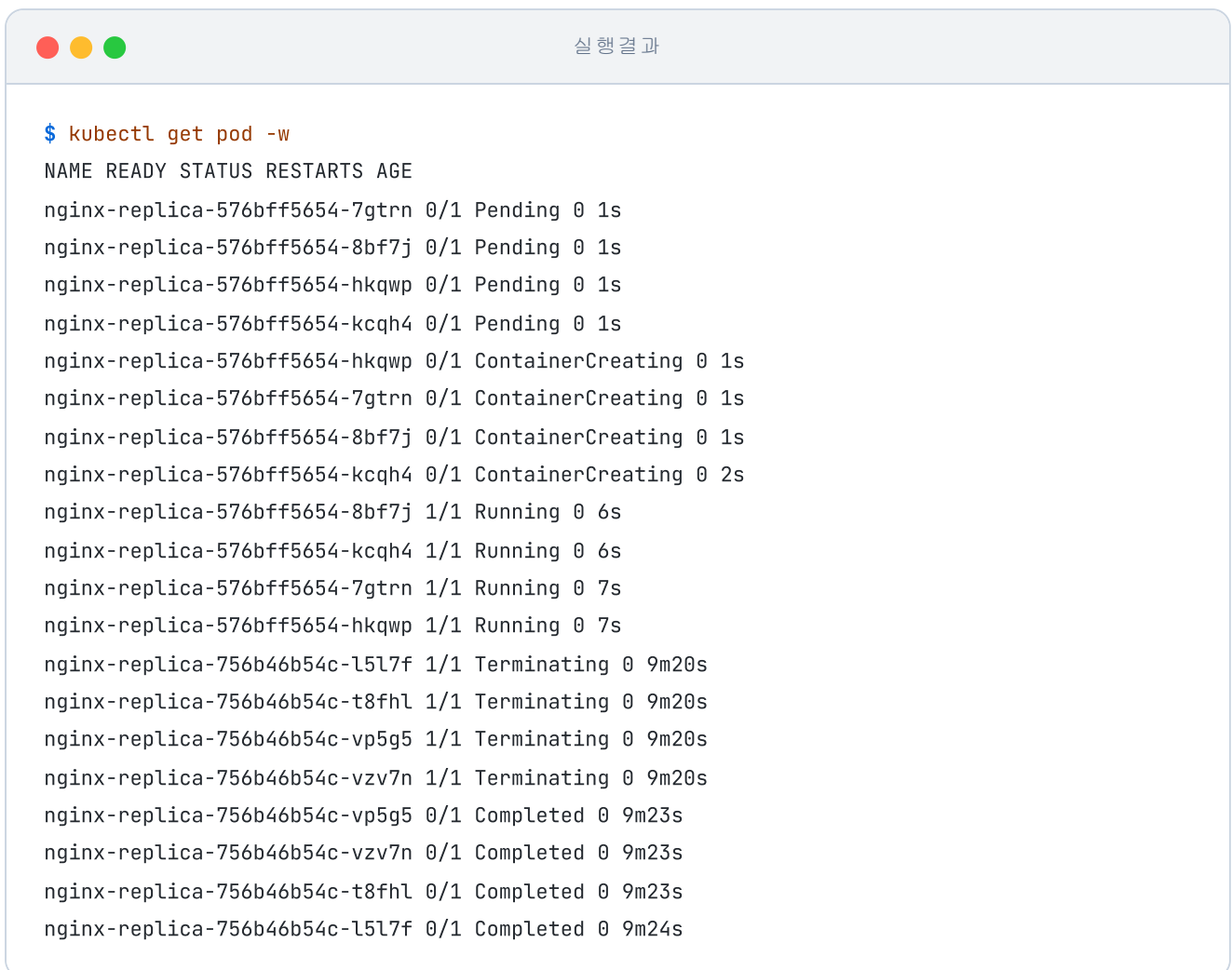
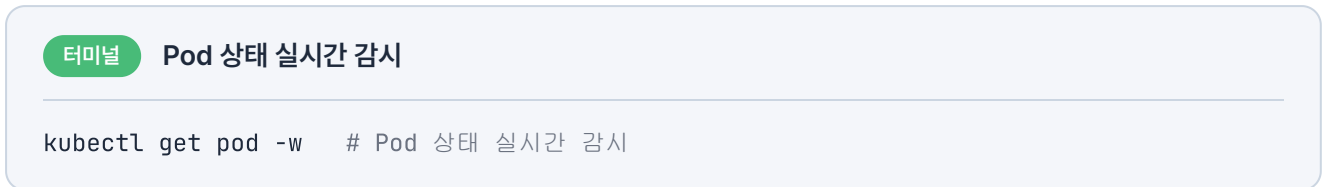


그림 4-25. 롤링 업데이트 진행 화면

출력에서 새 Pod는 **Pending → ContainerCreating → Running** 순서로 실행되고, 기존 Pod는 **Running → Terminating → Completed** 순서로 종료됩니다.

업데이트가 끝난 뒤 Deployment의 상세 정보를 확인하면 이미지가 `nginx:1.21` 버전으로 변경된 것을 볼 수 있습니다.

터미널 Deployment 상세 정보 조회

```
kubectl describe deployment nginx-replica # Deployment 상세 정보 조회
```

4.4.5 Rollback - 되돌리기

배포한 새 버전에서 뒤늦게 문제가 발견될 때가 있습니다. 그럴 때는 `kubectl rollout undo` 명령어로 이전 버전으로 되돌립니다.

터미널 롤백

```
kubectl rollout undo deployment/nginx-replica # 이전 버전으로 롤백
```

실행 결과

```
$ kubectl rollout undo deployment/nginx-replica
deployment.apps/nginx-replica rolled back
```

그림 4-26. Rollback 실행 결과

이렇게 쿠버네티스로 컨테이너를 운영하는 법을 익히고 나니, 문득 회의실에서의 그 질문이 떠올랐습니다.

'컨테이너가 죽거나 트래픽이 몰리면 어쩌나 막막했는데. 이제 쿠버네티스한테 맡기면 되겠다.'

다음 챕터에서는 Service와 Ingress로 외부와 연결되는 진입점을 만들어 보겠습니다.

이것만은 기억하자

- **Docker는 "지금 띄워라", Kubernetes는 "이 상태를 유지해라".** 선언형 관리의 핵심입니다. 원하는 상태를 YAML에 정의해 두면 쿠버네티스가 실제 상태를 계속 확인하며 그에 맞춥니다.
- **쿠버네티스는 컨트롤 플레인과 워커 노드로 이루어집니다.** 컨트롤 플레인이 워커 노드 위의 Pod를 관리합니다.
- **Pod는 쿠버네티스가 컨테이너를 다루는 최소 단위입니다.** 쿠버네티스는 컨테이너를 직접 다루지 않고 Pod 단위로 관리합니다.
- **운영에서는 Pod를 단독으로 만들지 않습니다.** 직접 만든 Pod는 장애가 나도 복구되지 않습니다. Deployment로 관리해야 자동 복구와 스케일링이 가능합니다.
- **롤링 업데이트로 무중단 배포를 합니다.** 새 Pod를 먼저 띄운 뒤 기존 Pod를 차례로 종료해, 서비스 중단 없이 버전을 올립니다.